# Building PES-VCS — A Version Control System from Scratch

# Operating Systems

## Orange problem solving(Unit 4)

Name :Vishwas V Angadi

SRN : PES1UG24AM416

Section : 4G

## Phase 1: Object Storage Foundation

## Screenshot 1A: Output of `./test_objects` showing all tests passing

```
vishwas@DESKTOP-CSLNSEH:~/os-u4-orange-problem$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

## Screenshot 1B: `find .pes/objects -type f` showing the shared directory structure

```
vishwas@DESKTOP-CSLNSEH:~/os-u4-orange-problem$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

# Phase 2: Tree Objects

# Screenshot 2A:Output of `./test_tree` showing all tests passing

```
vishwas@DESKTOP-CSLNSEH:~/os-u4-orange-problem$ gcc test_tree.c tree.c index.c object.c -o test_tree -lcrypto
vishwas@DESKTOP-CSLNSEH:~/os-u4-orange-problem$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

# Screenshot 2B:

```
vishwas@DESKTOP-CSLNSEH:~/os-u4-orange-problem$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
vishwas@DESKTOP-CSLNSEH:~/os-u4-orange-problem$
```

# Phase 3: The Index ( Staging area)

## Screenshot 3A:

```
vishwas@DESKTOP-CSLNSEH:~/os-u4-orange-problem$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.c
  untracked:  commit.h
  untracked:  commit.c
  untracked:  README.md
  untracked:  index.h
  untracked:  tree.h
  untracked:  test_objects.c
  untracked:  Makefile
  untracked:  index.c
  untracked:  pes.h
  untracked:  test_tree.c
  untracked:  object.c
  untracked:  test_sequence.sh
  untracked:  .gitignore
  untracked:  pes.c
```

## Screenshot 3B:

```
vishwas@DESKTOP-CSLNSEH:~/os-u4-orange-problem$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776338963 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776339084 6 file2.txt
```

# Phase 4: Commits and History

## Screenshot 4A:

```
vishwas@DESKTOP-CSLNSEH:~/os-u4-orange-problem$ ./pes log
commit 8a940dce9147b42dd066d599c2657e3c96e9b0ee51932b18ae9a3aa4a048c1f2
Author: Vishwas PES1UG24AM416
Date:   1776342914

    Add world

commit 151a52fec5d5f82061da1976c5d2b9a8214d904980d4cb974c0ff348449c3cee
Author: Vishwas PES1UG24AM416
Date:   1776342664

    Initial commit

commit 7c6e78b9e65f20c2e0f8c4ccdb9b190d663b78ffb9e630ed36f508f8e0023261
Author: PES User <pes@localhost>
Date:   1776342184

    Initial committial commiQ
```

## Screenshot 4B:

```
vishwas@DESKTOP-CSLNSEH:~/os-u4-orange-problem$ find .pes .type f | sort
find: '.type': No such file or directory
find: 'f': No such file or directory
.pes
.pes/HEAD
.pes/index
.pes/objects
.pes/objects/05
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/15
.pes/objects/15/1a52fec5d5f82061da1976c5d2b9a8214d904980d4cb974c0ff348449c3cee
.pes/objects/2c
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/66
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/7c
.pes/objects/7c/6e78b9e65f20c2e0f8c4ccdb9b190d663b78ffb9e630ed36f508f8e0023261
.pes/objects/8a
.pes/objects/8a/940dce9147b42dd066d599c2657e3c96e9b0ee51932b18ae9a3aa4a048c1f2
.pes/objects/e0
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/refs
.pes/refs/heads
.pes/refs/heads/main
```

Screenshot 4C:

```
vishwas@DESKTOP-CSLNSEH:~/os-u4-orange-problem$ cat .pes/refs/heads/main
8a940dce9147b42dd066d599c2657e3c96e9b0ee51932b18ae9a3aa4a048c1f2
vishwas@DESKTOP-CSLNSEH:~/os-u4-orange-problem$ cat .pes/HEAD
ref: refs/heads/main
```

## Phase 5 and 6: Analysis

## Branching and Checkout:

**Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?**

**Answer:** A branch in PES would be implemented as a file inside `.pes/refs/heads/` containing the commit hash.

To implement `pes checkout <branch>`, the following steps are required:

- Update `.pes/HEAD` to point to the selected branch reference.
- Read the commit hash stored in `.pes/refs/heads/<branch>`.
- Load the corresponding tree object from the object store.
- Update the working directory to match the tree structure by:
  - Creating files and directories
  - Writing file contents from blob objects
  - Removing files not present in the target branch

This operation is complex because it involves safely modifying the working directory while ensuring no data loss occurs.

**Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.**

**Answer:** To detect a dirty working directory:

- Compare each file in the index with the working directory using:
  - File size
  - Modification time (mtime)

Steps:

- If a file in the index has different metadata than the working directory → modified
- If file missing → deleted
- If differences exist and target branch has different version → conflict

Thus, checkout must refuse to proceed to avoid overwriting uncommitted changes.

**Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?**

**Answer:**

Detached HEAD occurs when `.pes/HEAD` contains a commit hash instead of a branch reference.

In this state:

- New commits are created but not linked to any branch
- These commits can become unreachable (lost)

Recovery:

- Use the commit hash to create a new branch:

  pes branch <name> <commit-hash>

- Or manually update `.pes/refs/heads/`

## Garbage Collection and Space Reclamation

**Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.**

**Answer:**

To find unreachable objects:

Algorithm:

1. Start from all branch heads (`.pes/refs/heads/*`)
2. Traverse commit graph:
   - Visit commits
   - Visit associated tree objects
   - Visit all blobs
3. Mark all visited objects as reachable

Data structure:

- Use a **hash set** to store visited hashes

Finally:

- Iterate through `.pes/objects`
- Delete objects not in reachable set

Estimation:

- 100,000 commits + trees + blobs ≈ several hundred thousand objects
- All must be traversed

**Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?**

**Answer:**

Running GC during commit is dangerous due to race conditions:

Example:

- Commit is creating new objects but hasn't updated references yet
- GC runs simultaneously
- GC does not see new objects as reachable
- GC deletes them
- Commit finishes → references missing objects → corruption

Git avoids this by:

- Using locks during operations
- Performing GC only when safe
- Using reference counting and packfiles